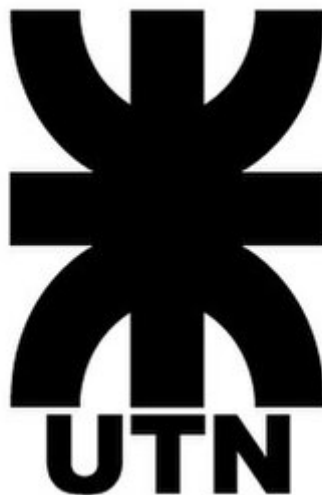


Trabajo Práctico final JAVA 2025

Sistema de Gestión de Proyectos

‘Planera’



Integrantes

52224 - Gambotto, Angel Uriel
52634 - Gregoret, Facundo Uriel
52420 - Lovatti, Francisco
52417 - Repupilli, Irina

Docentes

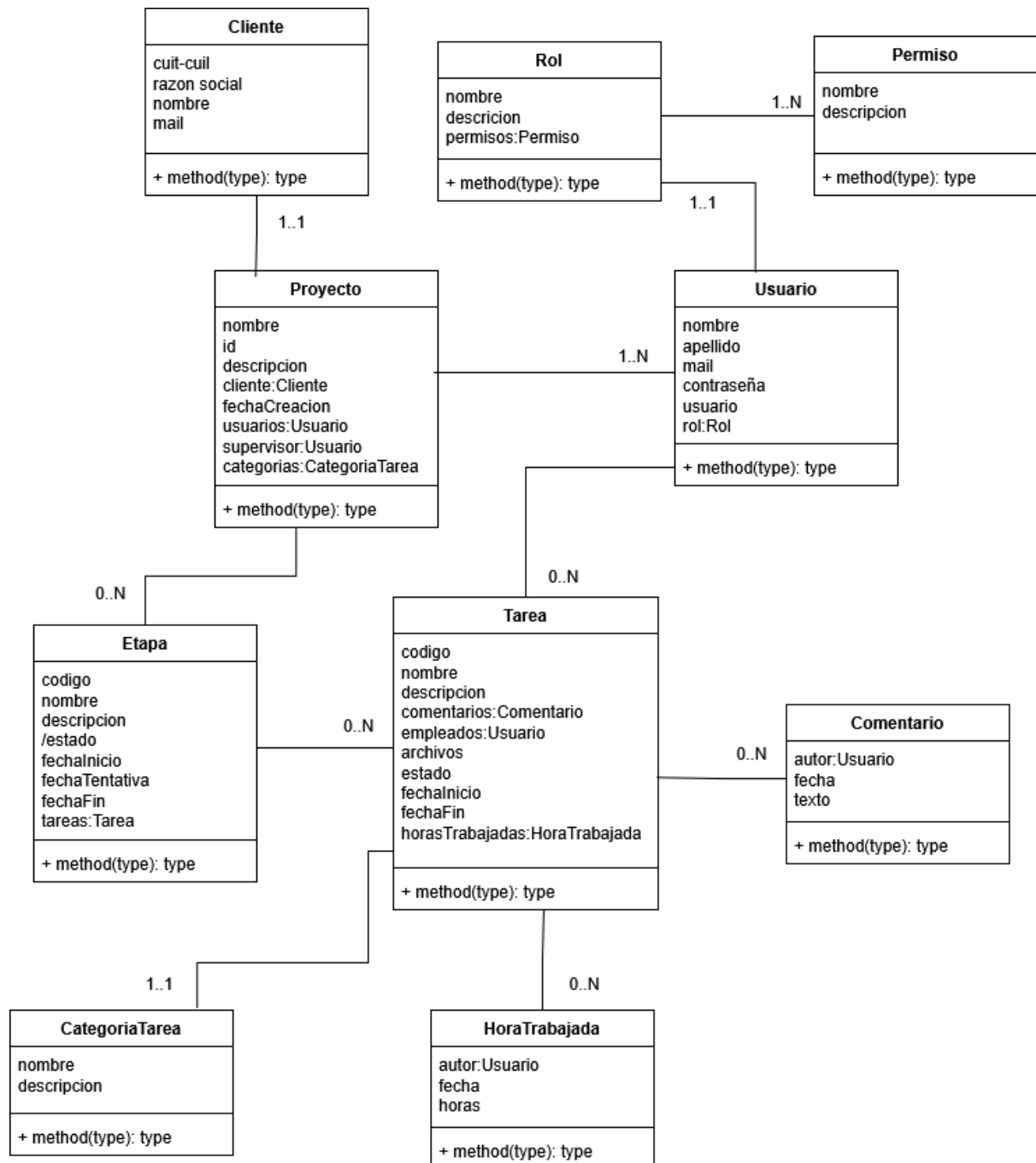
Meca, Adrian
Tabacman, Ricardo

Enunciado del Sistema

Este sistema permite gestionar proyectos colaborativos organizados por clientes, usuarios y tareas. Cada proyecto se asocia a un cliente, tiene un supervisor y múltiples usuarios involucrados, y se divide en etapas que contienen tareas específicas. Las tareas pueden tener empleados asignados, archivos, comentarios y registro de horas trabajadas, lo que permite un seguimiento detallado del avance. Además, las tareas se clasifican mediante categorías propias del proyecto, lo cual ayuda a mantener el orden y facilitar la planificación.

El sistema también contempla una estructura de roles y permisos para definir qué acciones puede realizar cada usuario, permitiendo diferenciar entre administradores, empleados y clientes. Gracias a esta organización, se facilita la supervisión del trabajo, el control del tiempo invertido, y la comunicación dentro de los equipos. En conjunto, el sistema ofrece una solución integral para la planificación, ejecución y seguimiento de proyectos en entornos colaborativos.

Modelo de datos



Requerimientos funcionales para regularidad

Requerimiento	Cantidad	Detalle
ABMC simple	4	1. ABMC - Usuario 2. ABMC - Proyecto 3. ABMC - Cliente 4. ABMC - Categoría de tarea
ABMC dependiente	2	1. ABMC - Etapa 2. ABMC - Tarea
CU NO-ABMC	2	1. CU - Ingresar horas trabajadas, solo pueden ser cargadas por usuarios asignados a la tarea. 2. CU - Visualizar seguimiento de proyecto y etapas, una etapa no puede finalizar si tiene tareas incompletas
Listado simple	3	1. Listado - Proyecto 2. Listado - Hora trabajada 3. Listado - Comentario

Requerimientos funcionales para Aprobación Directa

Requerimiento	Cantidad	Detalle
ABMC simple	4	1. ABMC - Usuario 2. ABMC - Proyecto 3. ABMC - Cliente 4. ABMC - Categoría de tarea
ABMC dependiente	3	1. ABMC - Etapa 2. ABMC - Tarea 3. ABMC - Comentario
CU "Complejo"(nivel resumen)	2	1. CU - Gestionar proyecto: desde la creación del proyecto hasta su finalización, pasando por la definición y seguimiento de etapas. 2. CU - Gestionar tarea: desde la creación hasta la finalización, pasando por la asignación de empleado, carga de comentarios y horas trabajadas
Listado complejo	1	1. Listado - Tarea. Filtrado por categoría, usuario,

		estado. 2. Listado - Usuario. Filtrado por rol y/o búsqueda por nombre y apellido.
Nivel de acceso	2	1. Administrador 2. Usuario avanzado(empleados) 3. Usuario (cliente)
Manejo de errores	-	-
Manejo de archivos	-	1. Carga de archivos a la descripción y comentarios de tareas
Envío de mails	-	1. Avisos de tarea asignada y cambios de estado de la misma 2. Aviso a usuarios cliente de cambio de estado en etapas.

Casos de Uso

CURS_1 - Gestionar proyectos de una organización.

Dimensiones de clasificación:

Nivel	Estructura	Alcance	Caja	Instanciación	Interacción
Resumen	Re-Estructurado	Sistema	Negra	Real	Semántica

Meta del CASO DE USO: Gestionar un proyecto de un cliente

Actor Primario: Supervisor (SU) **Otros:** Cliente (CLI), Empleado (EM)

PRECONDICIONES (de sistema): El proyecto se encuentra registrado con su supervisor asignado

El cliente se encuentra registrado

Los EM se encuentran registrados.

Las categorías de tareas están registradas.

DISPARADOR: SU debe gestionar un proyecto para su organización.

CAMINO BÁSICO:

1. SU asigna los EM que trabajarán en el proyecto invocando al CUU 1.1 Asignar empleados a un proyecto.
2. SU ingresa nueva etapa invocando al CUU 1.2 Registrar etapa.
Se repite el paso 2 las veces que se necesite.
3. SU ingresa nueva tarea con sus empleados a una etapa invocando al CUU 1.3 Gestionar tarea.
Se repite el paso 3 las veces que se necesite.
4. Cuando los EM comienzan a trabajar, SU cambia el estado de la etapa y del proyecto invocando al CUU 1.4 Actualizar estados en un proyecto.
Se repite el paso 4 las veces que se necesite.
5. Cuando se completan todas las etapas, el SU cambia el estado del proyecto a finalizado invocando al CUU 1.5 Finalizar un proyecto.

CAMINOS ALTERNATIVOS:

*<durante> CLI cancela el proyecto:

*.a.1 SU ingresa cancelación del proyecto invocando al CUU 1.6 Cancelar un proyecto. FIN CU

POSTCONDICIONES (de sistema):

Éxito: El proyecto está con estado finalizado.

Fracaso: El proyecto está registrado con estado cancelado (*.a.1).

Éxito alternativo: —

POSTCONDICIONES (de negocio):

Éxito: El CLI obtiene el proyecto finalizado.

Fracaso: El CLI cancela el proyecto.

Éxito alternativo: —

Reglas de Negocio relacionadas con el casos de uso:

Paso (s)	Regla de negocio
5	El proyecto no puede finalizarse si aún hay etapas sin finalizar.
3	Los empleados que se asignan a la tarea deben estar previamente asignados al proyecto.
4	La etapa no puede finalizarse si aún hay tareas sin finalizar.

CUU_1.3 Gestionar tarea

Dimensiones de clasificación:

Nivel	Estructura	Alcance	Caja	Instanciación	Interacción
Usuario	Sin estructurar	Sistema	Negra	Real	Semántica

Meta del CASO DE USO: Completar la tarea

Actor Primario: Supervisor (SU)

Otros: Empleado (EM)

PRECONDICIONES (de negocio): —

PRECONDICIONES (de sistema): Las categorías se encuentran registradas.

El proyecto y la etapa están registrados.

Los empleados están asignados al proyecto.

DISPARADOR: SU necesita una nueva tarea para llevar a cabo el proyecto.

CAMINO BÁSICO:

1. SU ingresa los datos de la tarea (nombre, descripción, estado, fecha de inicio, fecha de finalización, categoría). Sistema registra la tarea.
2. SU asigna EM a la tarea. Sistema registra los empleados para la tarea.
3. Cuando los EM comienzan a trabajar en la tarea, el SU actualiza el estado de la tarea a “En progreso”. Sistema actualiza el estado de la tarea.
4. Cuando los EM finalizan la tarea, el SU da por finalizada la tarea. Sistema registra el estado de la tarea como finalizada.

CAMINOS ALTERNATIVOS:

[2-4] SU cancela la tarea:

[2-4].a.1 SU actualiza el estado de la tarea a cancelada. Sistema registra. FIN CU.

POSTCONDICIONES (de sistema):

Éxito: La tarea está registrada como finalizada.

Fracaso: La tarea está registrada como cancelada.

Éxito alternativo: —

POSTCONDICIONES (de negocio):

Éxito: Los empleados completaron la tarea.

Fracaso: Supervisor decidió cancelar la tarea ([2-4].a.1).

Éxito alternativo: —

Diccionario de Datos:

Paso	Diccionario de Datos - Flujo	Tipo
1	nuevaTarea = id + nombre + descripcion + estado + fechaInicio + fechaFin + idEtapa + idCategoria	FE
2	empleadoAsignado = 1{idEmpleado}n	FE
3 y 4	actualizaTarea = estado	FE

CUU_1.7 Ingresar horas trabajadas.

Dimensiones de clasificación:

Nivel	Estructura	Alcance	Caja	Instanciación	Interacción
Usuario	Sin estructurar	Sistema	Negra	Real	Semántica

Meta del CASO DE USO: Registrar horas trabajadas en la tarea.

Actor Primario: Empleado (EM)

Otros: —

PRECONDICIONES (de negocio): —

PRECONDICIONES (de sistema): El proyecto y la etapa están registrados.

Los empleados están asignados a la tarea.

DISPARADOR: EM trabajó en la tarea.

CAMINO BÁSICO:

1. EM consulta las tareas a las que está asignado. Sistema informa.
2. EM selecciona la tarea en la que estuvo trabajando e ingresa la fecha y hora, el detalle de lo que hizo y las horas que trabajó. Sistema registra.

Se repite el paso 2 las veces que se necesite.

CAMINOS ALTERNATIVOS:

1<posterior> EM deja un comentario

1.a.1 EM selecciona la tarea en la que estuvo trabajando e ingresa la descripción del comentario y/o archivos. Sistema registra.

Se repite el paso 1.a.1 las veces que se necesite.

POSTCONDICIONES (de sistema):

Éxito: Las horas que trabajó el empleado en la tarea están registradas.

Fracaso: —

Éxito alternativo: Las horas que trabajó el empleado en la tarea y sus comentarios están registrados.

POSTCONDICIONES (de negocio):

Éxito: El empleado ingresó las horas trabajadas en la tarea.

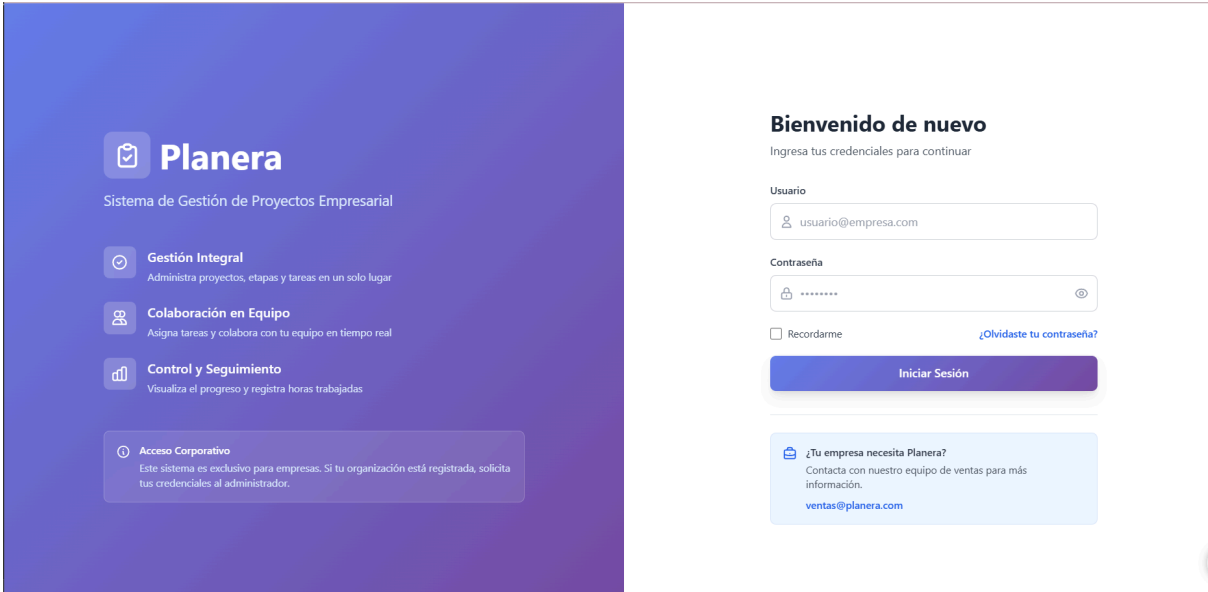
Fracaso: —

Éxito alternativo: El empleado ingresó las horas trabajadas y los comentarios en la tarea.

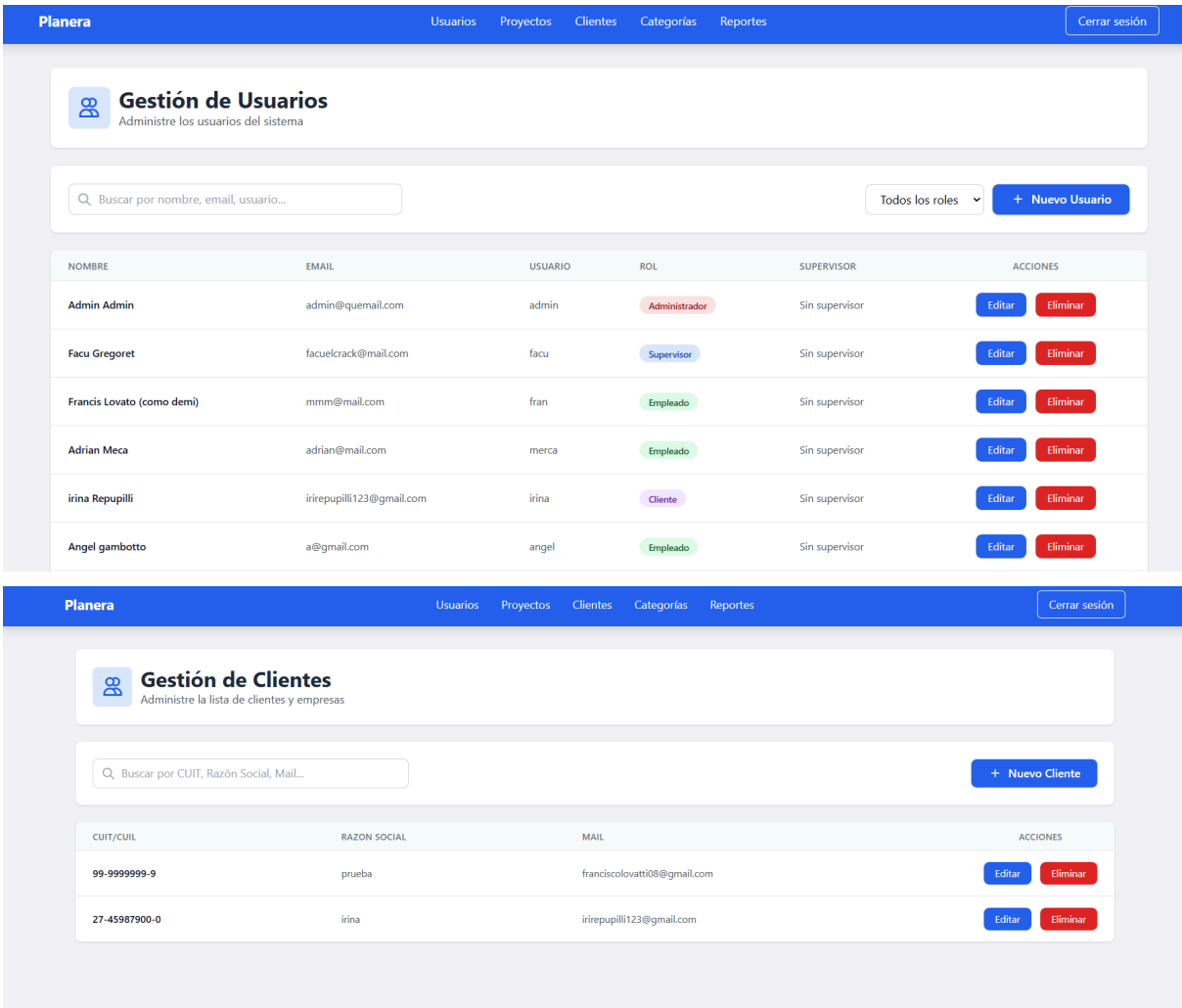
Diccionario de Datos:

Paso	Diccionario de Datos - Flujo	Tipo
2	horaTrabajada = idTarea + idEmpleado + detalle + fecha + cantidad	FE
1.a.1	comentario = idTarea + idEmpleado + detalle + 0{archivo}n + fecha	FE

Capturas de pantalla del sistema



USUARIO ADMINISTRADOR



Planera

UsuariosProyectosClientesCategoríasReportes

Cerrar sesión

Gestión de Categorías

Administre la lista de categorías de tareas

Q

Buscar por nombre o descripción

+ Nuevo Categoría

NOMBRE	DESCRIPCIÓN	ACCIONES
Frontend	tareas frontend	EditarEliminar
Software	estas son de sw	EditarEliminar

Planera

UsuariosProyectosClientesCategoríasReportes

Cerrar sesión

Gestión de Proyectos

Administre y visualice todos sus proyectos

Q

Buscar por nombre, descripción...

+ Nuevo Proyecto

Por Hacer2

Final

finalizando regularidad

prueba

Creación: 2025-11-18

2 usuarios asignados

Supervisor

FD

Testing Final

testeo general

irina

Creación: 2025-12-09

0 usuarios asignados

En Progreso0

Sin proyectos

Completados1

aprobacion

proyecto para aprobacionn

prueba

Creación: 2025-12-06

0 usuarios asignados

Supervisor

AG

Cancelados0

Sin proyectos

Planera

UsuariosProyectosClientesCategoríasReportes

Cerrar sesión

< Volver a Proyectos

Final

finalizando regularidad

To Do

Personas Asignadas

Francis Lovato (como demi)

Adrian Meca

+ Asignar Persona

+ Nueva Etapa

Editar Proyecto

Columns

Lista

Total Etapas2

Pendientes1

En Progreso0

Completadas1

ETAPA	ESTADO	FECHAS	ACCIONES
etapa 1 primera etapa	Done	Inicio: 18/11/2025 Tentativa: — Fin: —	EditarEliminar
etapa 2 blablabla	To Do	Inicio: 12/12/2025 Tentativa: — Fin: —	EditarEliminar

Planera

UsuariosProyectosClientesCategoríasReportes

Cerrar sesión

< Volver al Proyecto

etapa 2

To Do

+ Nueva Tarea

Editar Etapa

blablaba

Inicio: 12/12/2025Tentativa: 12/12/2025

Progreso General0%

0 de 1 tareas completadas

Total1

Por Hacer1

En Progreso0

Completadas0

Todos los estados

Todas las categorías

TablaKanban

TAREA	ESTADO	CATEGORÍA	FECHAS	ACCIONES
tarea 1 primera tarea	To Do	Frontend	Inicio: 12/12/2025 Fin: 12/12/2025	EditarEliminar

Planera

UsuariosProyectosClientesCategoríasReportes

Cerrar sesión

< Volver a Etapa

tarea 1

To Do

Editar Tarea

primera tarea

Inicio: 12/12/2025Fin: 12/12/2025

Personas Asignadas

+ Asignar Persona

Francis Lovato (como demi)

Horas TrabajadasComentarios

Registro de Horas

+ Registrar Horas

EMPLEADO	FECHA	DETALLE	HORAS
Admin, Admin	12/12/2025	testeo detalle	4 horas
Total de horas			4 horas

Horas TrabajadasComentarios

Comentarios

Escribe un comentario... (opcional si adjuntas archivos)

Adjuntar archivos (imágenes, PDFs, documentos...)

Elegir archivosSin archivos seleccionados

Puedes seleccionar varios archivos (máx 10 MB total recomendado)

Publicar Comentario

Admin, Admin

Primer comentario

12/12/2025 00:00

Eliminar

Reportes de Horas Trabajadas

Análisis detallado de tiempo por proyecto y colaborador

Filtros de Búsqueda

Fecha Desde

dd/mm/aaaa

Fecha Hasta

dd/mm/aaaa

Tipo de Reporte

Por Usuario

Generar Reporte

Total de Registros

2

Total de Horas

7

Promedio por Registro

3.5

Resultados del Reporte

USUARIO	HORAS TRABAJADAS
Angel gambotto	2 horas
Admin Admin	5 horas
Total	7 horas

Imprimir Reporte

USUARIO EMPLEADO

Planera

Mis Proyectos

Mis Tareas

Cerrar sesión

Mis Tareas

Tareas asignadas a Francis Lovato (como demi)

PROYECTO	ETAPA	TAREA	ESTADO	F. INICIO	F. FIN	ACCIONES
Testing Final	Etapa 1	tarea 1 primera tareaa	Done	2025-12-09	2025-12-09	Ver Tarea
aprobacion	cartelitos eliminar	Testing lmikm	Done	2025-12-11	2025-12-11	Ver Tarea
Final	etapa 2	tarea 1 primera tarea	To Do	2025-12-12	2025-12-12	Ver Tarea

USUARIO CLIENTE

PlaneraMis ProyectosCerrar sesión

**Portal del Cliente**
Seguimiento en tiempo real de sus proyectos

1

Bienvenido, irina

En este portal puede visualizar el estado y progreso de todos sus proyectos en curso. Cada proyecto muestra información actualizada sobre etapas, tareas completadas y horas dedicadas por nuestro equipo.

Testing FinalTo Do

testeo general

Supervisor: Angel gambotto

Progreso General

33%

1 de 3 etapas completadas

Etapas

3

Tareas

2

Horas

0

Ver Detalles del Proyecto >

Testing FinalTo Do

testeo general

Supervisor: Angel gambotto Inicio: 09/12/2025

Progreso General del Proyecto

33%

1 de 3 etapas completadas

Etapas Totales

3

En Progreso

1

Tareas Totales

2

Horas Dedicadas

0

Estado de Etapas



Por Iniciar

1

En Progreso

1

Completadas

1

Desglose de Etapas

Etapa 1 Done

primera etapa

100% completado

1 de 1 tareas completadas Inicio: 09/12/2025

etapa 2 In Progress

segunda etapa

0% completado

0 de 1 tareas completadas Inicio: 09/12/2025

etapa 3 To Do

ftbbb

0% completado

0 de 0 tareas completadas Inicio: 12/12/2025

Código del sistema

Links al repositorio git con el código completo: <https://github.com/angelgambotto/tp-java>

CUU Gestionar Tarea

TareaServlet.java


```
protected void doGet(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {
    try {
```

```
        Usuario usuario = (Usuario)
request.getSession().getAttribute("usuario");
        String rol = usuario != null ? usuario.getRol().toLowerCase() : "";
        request.setAttribute("usuario", usuario);

        boolean esAdmin = "administrador".equals(rol);
        boolean esEmpleado = "empleado".equals(rol);
        // Acción por defecto según rol
        String action = request.getParameter("action");
        if (action == null) {
            action = esAdmin ? "list" : "mis-tareas";
        }
        switch(action) {
            case "list":
                //...
                break;
            case "new":
                idEtapa = Integer.parseInt(request.getParameter("idEtapa"));
                String estadoEtapa=edao.getOne(idEtapa).getEstado();

                List<Usuario> usuariosDisponibles = udao.getAll();
                List<CategoriaTarea> categorias = cdao.getAll();

                int idEt=Integer.parseInt(request.getParameter("idEtapa"));
                List<Tarea> tar=tdao.getByEtapald(idEt);
                request.setAttribute("tareas", tar);

                Etapa et=edao.getOne(idEtapa);
                int idProNew = et.getIdProyecto();
                Proyecto proNew = cargarProSeguro(request, idProNew);
                request.setAttribute("proyecto", proNew);
                if ("Done".equals(estadoEtapa)) {
                    request.setAttribute("error", "No se pueden agregar tareas en una etapa
finalizada.");
                    request.getRequestDispatcher("etapas/unaEtapa.jsp").forward(request,
response);
                }
                request.setAttribute("etapa", et);
                request.setAttribute("EtapaFinalizada", estadoEtapa.equals("Done"));
                request.setAttribute("usuarios", usuariosDisponibles);
                request.setAttribute("categorias", categorias);
                request.setAttribute("idEtapa", idEtapa);
                request.setAttribute("abrirModalTarea", true);
                request.getRequestDispatcher("etapas/unaEtapa.jsp").forward(request, response);
            }
    }
}
```

```

        break;
    case "edit":
        //...
break;
    case "mis-tareas":
        //...
        break;
    case "detalle":
        String tab = request.getParameter("tab");
        int idE=Integer.parseInt(request.getParameter("idEtapa"));
        int idT=Integer.parseInt(request.getParameter("idTarea"));
        Etapa etapa = edao.getOne(idE);
        int idPDetalle = etapa.getIdProyecto();
        Proyecto proDetalle = cargarProSeguro(request, idPDetalle);

        Tarea tareaDetalle = tdao.getOne(idT);
        List<Usuario> asignados = tdao.getUsuariosAsignados(idT);
        List<Usuario> disponibles = pdao.getUsuariosAsignados(etapa.getIdProyecto());
        List<Comentario> comentarios = comdao.getAllByIdTarea(tareaDetalle.getId());
        List<HoraTrabajada> horas = htdao.getAllByIdTarea(tareaDetalle.getId());
        List<Usuario> administradores = udao.getPorRol("Administrador");

        request.setAttribute("administradores", administradores);
        request.setAttribute("etapa", etapa);
        request.setAttribute("tarea", tareaDetalle);
        request.setAttribute("empleadosAsignados", asignados);
        request.setAttribute("empleadosDisponibles", disponibles);
        request.setAttribute("comentarios", comentarios);
        request.setAttribute("horas", horas);
        request.setAttribute("usuario", usuario);
        request.setAttribute("proyecto", proDetalle);
        request.setAttribute("tab", tab);

        request.getRequestDispatcher("/tareas/unaTarea.jsp").forward(request, response);
        break;
    case "delete":
        //...
        break;
} //cierra switch
} catch(DAOException e) {
    throw new ServletException("Error en tareasservlet get");
}

}
/**
 * @see HttpServlet#doPost(HttpServletRequest request, HttpServletResponse
response)
 */

```

```
protected void doPost(HttpServletRequest request, HttpServletResponse response)
throws ServletException, IOException {
```

```
    String action=request.getParameter("action");
    switch(action) {
    case "insert":
        insertarTarea(request,response);
        break;
    case "update":
        actualizarTarea(request,response);
        break;
    case "confirmDelete":
        borrarTarea(request,response);
        break;
    case "asignar":
        asignarUsuarios(request,response);
    }
}
```

```
private void insertarTarea(HttpServletRequest request,HttpServletResponse
response) throws IOException{
    try{Tarea tarea=new Tarea();
    tarea.setNombre(request.getParameter("nombre"));
    tarea.setDescripcion(request.getParameter("descripcion"));
    tarea.setEstado(request.getParameter("estado"));
    tarea.setFechaInicio(Date.valueOf(request.getParameter("fechaInicio")));
    String fechaFinStr=request.getParameter("fechaFin");
    if(fechaFinStr==null||fechaFinStr.trim().isEmpty()){
        tarea.setFechaFin(null);
    }
    else {
        tarea.setFechaFin(Date.valueOf(fechaFinStr));
    }
    tarea.setIdEtapa(Integer.parseInt(request.getParameter("idEtapa")));
    tarea.setIdCategoria(Integer.parseInt(request.getParameter("idCategoria")));
    Etapa e=edao.getOne(Integer.parseInt(request.getParameter("idEtapa")));
    String estadoEtapa=e.getEstado();
    if(estadoEtapa.equals("Done")){
        request.setAttribute("error", "No se puede crear una tarea cuando la
etapa está finalizada");
        request.setAttribute("tarefas", tdao.getByEtapaId(tarea.getIdEtapa()));
        request.setAttribute("categorias", cdao.getAll());
        request.setAttribute("etapa", edao.getOne(tarea.getIdEtapa()));
        request.getRequestDispatcher("etapas/unaEtapa.jsp");
    }
    integer idTarea=tdao.insert(tarea);
    response.sendRedirect("TareaServlet?action=detalle&idTarea="+idTarea+"&idEtapa=" +
tarea.getIdEtapa());
```

```

    }
    catch(DAOException e) {
        request.setAttribute("Error al insertar tarea: ", e);
    }
}

private void asignarUsuarios(HttpServletRequest request, HttpServletResponse
response) throws IOException, ServletException {
    try {
        int id = request.getParameter("idTarea") == null ||
request.getParameter("idTarea").isEmpty()
        ? 0 : Integer.parseInt(request.getParameter("idTarea"));
        Tarea tarea = tdao.getOne(id);
        // Usuarios que ya estaban asignados antes de guardar
        List<Integer> usuariosAntes = tdao.getUsuariosAsignados(id)
            .stream()
            .map(Usuario::getId)
            .toList();
        String[] usuariosForm = request.getParameterValues("usuarios");
        List<Integer> usuariosSeleccionados = new ArrayList<>();
        if (usuariosForm != null) {
            for (String idU : usuariosForm) {
                usuariosSeleccionados.add(Integer.parseInt(idU));
            }
        }
        tdao.asignarEmpleados(id, usuariosSeleccionados);
        // Detectar solo los nuevos usuarios asignados
        List<Integer> usuariosNuevos = usuariosSeleccionados.stream()
            .filter(u -> !usuariosAntes.contains(u))
            .toList();
        // ===== ENVÍO DE MAIL SOLO A NUEVOS =====
        for (Integer idUser : usuariosNuevos) {
            Usuario u = udao.getOne(idUser);
            if (u != null) {
                String asunto = "Nueva tarea asignada: " + tarea.getNombre();
                String cuerpo =
                    "<div style='font-family: Arial, sans-serif; background:#f4f4f4;
padding:20px;'>"
                    + " <div style='max-width:600px; margin:auto; background:#ffffff;
padding:20px; border-radius:8px;'>"
                    + " <h2 style='color:#1a73e8;'>Nueva tarea asignada</h2>"
                    + " <p>Hola <strong> " + u.getNombreCompleto() + "</strong>, </p>"
                    + " <p>Se te ha asignado una nueva tarea.</p>"
                    + " <hr style='border:none; border-top:1px solid #ddd; margin:20px 0;'>"
                    + " <h3 style='color:#444;'>Detalles</h3>"
                    + " <p><strong>Tarea:</strong> " + tarea.getNombre() + "<br>"
                    + " <strong>Descripción:</strong> " + tarea.getDescripcion() + "<br>"
                    + " <strong>Fecha inicio:</strong> " + tarea.getFechaInicio() + "<br>"

```

```

        + "    <strong>Fecha fin:</strong> " + tarea.getFechaFin() + "</p>"
        + " </div>"
        + "</div>";
    try {
        emailService.enviar(u.getMail(), asunto, cuerpo);
    } catch (Exception ex) {
        ex.printStackTrace();
        System.out.println("[ERROR] No se envió mail a " + u.getMail());
    }
}

}

response.sendRedirect("TareaServlet?action=detalle&idEtapa=" + tarea.getIdEtapa()
+ "&idTarea=" + tarea.getId());
    } catch (Exception e) { // <--- Captura cualquier error
        throw new ServletException("Error en asignarUsuarios", e);
    }
}
}

```

TareaDAO.java

```
public class TareaDAO {

    public Integer insert(Tarea tarea) throws DAOException {
        String sql = "INSERT INTO tarea (nombre, descripcion, estado, fechaInicio,
        fechaFin, idEtapa, idCategoria) VALUES (?, ?, ?, ?, ?, ?, ?)";

        Connection con = null;
        PreparedStatement ps = null;

        ResultSet rs = null;
        Integer id=null;
        try {
            con = ConexionDB.getConexion();
            con.setAutoCommit(false);

            ps = con.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS);
            ps.setString(1, tarea.getNombre());
            ps.setString(2, tarea.getDescripcion());
            ps.setString(3, tarea.getEstado());
            ps.setDate(4, new Date(tarea.getFechaInicio().getTime()));
            if (tarea.getFechaFin() != null) {
                ps.setDate(5, new Date(tarea.getFechaFin().getTime()));
            } else {
                ps.setNull(5, java.sql.Types.DATE);
            }
            ps.setInt(6, tarea.getIdEtapa());
            ps.setInt(7, tarea.getIdCategoria());
            ps.executeUpdate();

            rs = ps.getGeneratedKeys();
            if (rs.next()) {
                id=rs.getInt(1);
                tarea.setId(id);
            }
            con.commit();
            return id;
        } catch (SQLException e) {

            if (con != null) {
                try {
                    con.rollback();
                } catch (SQLException ex) {
                    ex.printStackTrace();
                }
            }
        }
    }
}
```

```

    }
    e.printStackTrace();
    throw new DAOException("Error al agregar la tarea: " + tarea.getNombre(), e);
} finally {

    try {
        if (rs != null) rs.close();
        if (ps != null) ps.close();

        if (con != null) {
            con.setAutoCommit(true);
            con.close();
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

}

public List<Usuario> getUsuariosAsignados(int idTarea) throws DAOException {
    List<Usuario> usuarios = new ArrayList<>();

    String sql = "SELECT u.id, u.nombre, u.apellido, u.mail FROM usuario u
INNER JOIN tarea_usuario tu ON tu.idEmpleado = u.id WHERE tu.idTarea = ?";

    try (Connection con = ConexionDB.getConexion();
        PreparedStatement ps = con.prepareStatement(sql)) {

        ps.setInt(1, idTarea);

        try (ResultSet rs = ps.executeQuery()) {

            while (rs.next()) {
                Usuario u = new Usuario();
                u.setId(rs.getInt("id"));
                u.setNombre(rs.getString("nombre"));
                u.setApellido(rs.getString("apellido"));
                u.setMail(rs.getString("mail"));

                usuarios.add(u);
            }
        } catch (SQLException e) {
            throw new DAOException("Error obteniendo usuarios asignados a la
tarea", e);
        }

        return usuarios;
    }
}

```

```

    }

    public void delete(int id) throws DAOException {
        String sqlHoras = "DELETE FROM hora_trabajada WHERE idTarea =
?";

        String sqlComentario = "DELETE FROM comentario WHERE idTarea
= ?";

        String sqlTareaUsuario = "DELETE FROM tarea_usuario WHERE
idTarea = ?";

        String sqlTarea = "DELETE FROM tarea WHERE id = ?";

        try (Connection con = ConexionDB.getConexion()) {

            try (PreparedStatement ps2 = con.prepareStatement(sqlHoras)) {
                ps2.setInt(1, id);
                ps2.executeUpdate();
            }
            try (PreparedStatement ps3 = con.prepareStatement(sqlComentario))
{
                ps3.setInt(1, id);
                ps3.executeUpdate();
            }

            try (PreparedStatement ps1 = con.prepareStatement(sqlTareaUsuario)) {
                ps1.setInt(1, id);
                ps1.executeUpdate();
            }

            // Finalmente borrar la tarea
            try (PreparedStatement ps4 = con.prepareStatement(sqlTarea)) {
                ps4.setInt(1, id);
                ps4.executeUpdate();
            }

            } catch (SQLException e) {
                throw new DAOException("Error al eliminar la tarea con id: "+ id, e);
            }
        }

        public List<Tarea> getByEtapald(int idEtapa) throws DAOException {
            String sql = "SELECT * FROM tarea WHERE idEtapa = ? ORDER BY
fechalnicio ASC"; // Orden lógico por fecha
            List<Tarea> tareas = new ArrayList<>();
            try (Connection con = ConexionDB.getConexion());
            PreparedStatement ps = con.prepareStatement(sql)) {
                ps.setInt(1, idEtapa);

```



```

        ResultSet rs = ps.executeQuery();
        while (rs.next()) {
            Tarea tarea = new Tarea();
            tarea.setId(rs.getInt("id"));
            tarea.setNombre(rs.getString("nombre"));
            tarea.setDescripcion(rs.getString("descripcion"));
            tarea.setEstado(rs.getString("estado"));
            tarea.setFechaInicio(rs.getDate("fechaInicio"));
            tarea.setFechaFin(rs.getDate("fechaFin"));
            tarea.setIdEtapa(rs.getInt("idEtapa")); // ID para referencia, sin cargar

Etapa completa

            // Cargar CategoriaTarea
            //CategoriaTareaDAO cdao = new CategoriaTareaDAO();
            //CategoriaTarea cat = cdao.getById();
            tarea.setIdCategoria(rs.getInt("idCategoria"));

            tareas.add(tarea);
        }
    } catch (SQLException e) {
        throw new DAOException("Error al obtener tareas para etapa id: " +
idEtapa, e);
    }
    return tareas;
}

public boolean tieneTareasIncompletas (int idEtapa) throws DAOException {
    String sql="SELECT COUNT(*) from tarea where idEtapa=? and
estado<>'Done'";

    try {
        Connection con=ConexionDB.getConexion();
        PreparedStatement ps=con.prepareStatement(sql);
        ps.setInt(1, idEtapa);
        ResultSet rs=ps.executeQuery();
        if(rs.next()) {
            return rs.getInt(1)>0;
        }

    }catch (SQLException e) {
        throw new DAOException("Error al obtener tareas incompletas
de etapa con id: "+idEtapa,e);
    }

    return false;
}

public void asignarEmpleados(int idT, List<Integer> asignados) throws
DAOException {
    String sqlActuales = "SELECT * from tarea_usuario where idTarea = ?";

```

```

        String sqlAsignar = "INSERT INTO tarea_usuario (idTarea, idEmpleado)
VALUES (?,?)";
        String sqlBaja = "DELETE FROM tarea_usuario where idEmpleado = ? and
idTarea = ?";

        try {
            Connection con=ConexionDB.getConexion();
            PreparedStatement psUsuariosAct =
con.prepareStatement(sqlActuales);
            psUsuariosAct.setInt(1, idT);
            ResultSet rs = psUsuariosAct.executeQuery();
            List<Integer> usuariosActualesBD = new ArrayList<>();

            while (rs.next()) {
                usuariosActualesBD.add(rs.getInt("idEmpleado"));
            }

            PreparedStatement psInsert = con.prepareStatement(sqlAsignar);
            for (Integer u : asignados) {
                if (!usuariosActualesBD.contains(u)) {
                    psInsert.setInt(1, idT);
                    psInsert.setInt(2, u);
                    psInsert.executeUpdate();
                }
            }

            PreparedStatement psDelete = con.prepareStatement(sqlBaja);
            for (Integer u : usuariosActualesBD) {
                if (!asignados.contains(u)) {
                    psDelete.setInt(1, u);
                    psDelete.setInt(2, idT);
                    psDelete.executeUpdate();
                }
            }

        } catch (SQLException e) {
            throw new DAOException("Error actualizando proyecto", e);
        }
    }
}

```

```
<body class="bg-gray-100 font-sans">
<jsp:include page="../header.jsp" />

<div class="container mx-auto px-4 sm:px-6 lg:px-8 py-4 sm:py-6 lg:py-8">
  <!-- BOTÓN VOLVER -->
  <div class="mb-4">
    <a href="TareaServlet?action=list&idEtapa=<%= tarea.getIdEtapa() %>"
      class="flex items-center gap-2 text-gray-600 hover:text-gray-800 font-medium
transition">

      <svg class="w-5 h-5" fill="none" stroke="currentColor" viewBox="0 0 24 24">
        <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M15
19 7 7 7"></path>
      </svg>

      Volver a Etapa
    </a>
  </div>
  <!-- HEADER DE LA TAREA -->
  <% if (tarea != null) { %>
    <div class="bg-white rounded-lg shadow p-4 sm:p-6 mb-4 sm:mb-6">
      <div class="flex flex-col lg:flex-row lg:items-start lg:justify-between gap-4 mb-4">
        <div class="flex-1">
          <div class="flex flex-wrap items-center gap-2 mb-2">
            <h1 class="text-xl sm:text-2xl font-bold text-gray-800"><%=
tarea.getNombre() %></h1>
            <span class="px-3 py-1 text-xs font-semibold rounded-full
              <%= "To Do".equals(tarea.getEstado()) ? "bg-yellow-100 text-yellow-800" :
                "In Progress".equals(tarea.getEstado()) ? "bg-blue-100 text-blue-800" :
                "Done".equals(tarea.getEstado()) ? "bg-green-100 text-green-800" :
                "bg-red-100 text-red-800" %><%=
              tarea.getEstado() %>
            </span>
          </div>
          <p class="text-sm sm:text-base text-gray-600 mb-3"><%= tarea.getDescripcion()
%></p>

          <!-- Fechas -->
          <div class="flex flex-wrap gap-4 text-sm text-gray-600">
            <div class="flex items-center">
              <svg class="w-4 h-4 mr-1.5 text-gray-400" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
                <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
d="M8 7V3m8 4V3m-9 8h10M5 21h14a2 2 0 02-2V7a2 2 0 02-2-2H5a2 2 0 02-2 2v12a2 2
0 02 2z"></path>
              </svg>
```

```

        <span class="font-medium">Inicio:</span>
        <span class="ml-1"><%= tarea.getFechaInicio() != null ?
sdf.format(tarea.getFechaInicio()) : "—" %></span>
    </div>
    <div class="flex items-center">
        <svg class="w-4 h-4 mr-1.5 text-gray-400" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
            <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
d="M8 7V3m8 4V3m-9 8h10M5 21h14a2 2 0 02-2V7a2 2 0 00-2-2H5a2 2 0 00-2 2v12a2 2
0 002 2z"></path>
        </svg>
        <span class="font-medium">Fin:</span>
        <span class="ml-1"><%= tarea.getFechaFin() != null ?
sdf.format(tarea.getFechaFin()) : "—" %></span>
    </div>
</div>
<!-- Botón de editar -->
<div class="flex-shrink-0">
    <% if (rol.equalsIgnoreCase("administrador")) ||
        usuarioActual.getId() == pro.getSupervisor().getId()) { %>

        <a href="TareaServlet?action=edit&idTarea=<%= tarea.getId() %>"
        class="inline-block bg-blue-600 hover:bg-blue-700 text-white font-medium py-2
px-4 sm:px-6 rounded-lg shadow transition text-center">
            <svg class="inline w-4 h-4 mr-1" fill="none" stroke="currentColor" viewBox="0
0 24 24">
                <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
d="M11 5H6a2 2 0 00-2 2v11a2 2 0 002 2h11a2 2 0 002-2v-5m-1.414-9.414a2 2 0 112.828
2.828L11.828 15H9v-2.828l8.586-8.586z"></path>
            </svg>
            Editar Tarea
        </a>
        <% } %>
    </div>
</div>
<!-- EMPLEADOS ASIGNADOS -->
<div class="border-t pt-4">
    <div class="flex flex-col sm:flex-row sm:items-center sm:justify-between gap-3
mb-3">
        <h3 class="text-sm font-semibold text-gray-700 flex items-center">
            <svg class="w-5 h-5 mr-2 text-gray-500" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
                <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
d="M12 4.354a4 4 0 110 5.292M15 21H3v-1a6 6 0 0112 0v1zm0 0h6v-1a6 6 0
00-9-5.197M13 7a4 4 0 11-8 0 4 4 0 018 0z"></path>
            </svg>
            Personas Asignadas
        </h3>
    </div>
    </div>

```

```

</h3>
<% if (rol.equalsIgnoreCase("administrador") ||
    usuarioActual.getId() == pro.getSupervisor().getId()) { %>

    <button onclick="toggleModal('modalAsignarEmpleado')"
        class="text-sm bg-green-600 hover:bg-green-700 text-white font-medium
py-1.5 px-4 rounded-lg transition">
        + Asignar Persona
    </button>
    <% } %>
</div>
<div class="flex flex-wrap gap-2">
    <% if (empleadosAsignados != null && !empleadosAsignados.isEmpty()) {
        for (Usuario emp : empleadosAsignados) {
            String[] colores = {"bg-purple-400", "bg-blue-400", "bg-indigo-400"};
            String color = colores[emp.getId() % colores.length];
        %>
            <div class="flex items-center gap-2 <%= color %> text-white px-3 py-1.5
rounded-full text-sm font-medium">
                <span><%= emp.getNombre() %> <%= emp.getApellido() %></span>
            </div>
        <% }
    } else { %>
        <p class="text-sm text-gray-500 italic">No hay personas asignadas aún</p>
    <% } %>
</div>
</div>
</div>
<% } else { %>
    <div class="bg-red-100 border border-red-400 text-red-700 px-4 py-3 rounded mb-4
sm:mb-6">
        Tarea no encontrada
    </div>
    <% } %>
<%
// Variable que indica si el usuario puede ver el contenido sensible
boolean puedeVerContenido = false;
// 1. Es administrador?
if ("administrador".equalsIgnoreCase(rol) ||
    usuarioActual.getId() == pro.getSupervisor().getId()) {
    puedeVerContenido = true;
}
// 2. Si no es admin, revisamos si está asignado a la tarea
else if (empleadosAsignados != null && usuarioActual != null) {
    for (Usuario emp : empleadosAsignados) {
        if (emp.getId() == usuarioActual.getId()) {
            puedeVerContenido = true;
            break;

```

```

    }
  }
}
%>
<% if (puedeVerContenido) { %>
  <!-- TABS -->
  <div class="bg-white rounded-lg shadow mb-6">
    <div class="border-b overflow-x-auto">
      <nav class="flex min-w-max sm:min-w-0">
        <button onclick="cambiarTab('horas')" id="tabHoras"
          class="tab-active px-4 sm:px-6 py-3 text-sm font-medium text-gray-700
whitespace-nowrap transition">
          <svg class="inline w-4 h-4 mr-1.5" fill="none" stroke="currentColor" viewBox="0
0 24 24">
            <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M12
8v4l3 3m6-3a9 9 0 11-18 0 9 9 0 0118 0z"></path>
          </svg>
          Horas Trabajadas
        </button>
        <button onclick="cambiarTab('comentarios')" id="tabComentarios"
          class="px-4 sm:px-6 py-3 text-sm font-medium text-gray-700
hover:text-blue-600 whitespace-nowrap transition">
          <svg class="inline w-4 h-4 mr-1.5" fill="none" stroke="currentColor" viewBox="0
0 24 24">
            <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M8
12h.01M12 12h.01M16 12h.01M21 12c0 4.418-4.03 8-9 8a9.863 9.863 0 01-4.255-.949L3
20l1.395-3.72C3.512 15.042 3 13.574 3 12c0-4.418 4.03-8 9-8s9 3.582 9 8z"></path>
          </svg>
          Comentarios
        </button>
      </nav>
    </div>
    <!-- CONTENIDO DE HORAS TRABAJADAS -->
    <div id="contentHoras" class="p-4 sm:p-6">
      <div class="flex flex-col sm:flex-row sm:items-center sm:justify-between gap-3
mb-4">
        <h2 class="text-lg font-semibold text-gray-800">Registro de Horas</h2>
        <% if (rol.equalsIgnoreCase("empleado") || rol.equalsIgnoreCase("administrador"))
{ %>
          <a href="HoraTrabajadaServlet?action=new&idTarea=<%= tarea.getId()
%>&idEmpleado=<%= usuarioActual.getId() %>&origin=unaTarea"
            class="bg-blue-600 hover:bg-blue-700 text-white font-medium py-2 px-4
rounded-lg transition text-sm">
            + Registrar Horas
          </a>
        <% } %>
      </div>

```

```

<% if (horas == null || horas.isEmpty()) { %>
    <div class="text-center py-12 text-gray-400">
        <svg class="mx-auto h-12 w-12 mb-3" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
            <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M12
8v4l3 3m6-3a9 9 0 11-18 0 9 9 0 0118 0z"></path>
        </svg>
        <p class="text-sm">No hay horas registradas aún</p>
    </div>
<% } else { %>
    <!-- Vista móvil: Cards -->
    <div class="block lg:hidden space-y-3">
        <%
            int totalHoras = 0;
            for (HoraTrabajada h : horas) {
                totalHoras += h.getCantidad();
                int idEmp = h.getIdEmpleado();
                String nombreEmp = "NN";
                if (empleadosAsignados != null) {
                    for (Usuario e : empleadosAsignados) {
                        if (e.getId() == idEmp) {
                            nombreEmp = e.getApellido() + ", "
+e.getNombre();
                            break;
                        }
                    }
                }
            %>
            <div class="bg-gray-50 rounded-lg p-4 border border-gray-200">
                <div class="flex justify-between items-start mb-2">
                    <div>
                        <p class="font-semibold text-gray-800"><%= nombreEmp %></p>
                        <p class="text-xs text-gray-500"><%= h.getFecha() != null ?
sdf.format(h.getFecha()) : "—" %></p>
                    </div>
                    <span class="bg-blue-100 text-blue-800 text-sm font-bold px-3 py-1
rounded-full">
                        <%= h.getCantidad() %>h
                    </span>
                </div>
            </div>
        <% } %>
        <div class="bg-blue-50 rounded-lg p-4 border-2 border-blue-200">
            <p class="text-sm text-gray-600">Total de horas:</p>
            <p class="text-2xl font-bold text-blue-600"><%= totalHoras %> horas</p>
        </div>
    </div>
    <!-- Vista desktop: Tabla -->

```

```

<div class="hidden lg:block overflow-x-auto">
  <table class="w-full">
    <thead class="bg-gray-50 border-b">
      <tr>
        <th class="px-6 py-3 text-left text-xs font-medium text-gray-500 uppercase">Empleado</th>
        <th class="px-6 py-3 text-left text-xs font-medium text-gray-500 uppercase">Fecha</th>
        <th class="px-6 py-3 text-left text-xs font-medium text-gray-500 uppercase">Detalle</th>
        <th class="px-6 py-3 text-left text-xs font-medium text-gray-500 uppercase">Horas</th>
      </tr>
    </thead>
    <tbody class="divide-y divide-gray-200">
      <%
        totalHoras = 0;
        for (HoraTrabajada h : horas) {
          totalHoras += h.getCantidad();
          int idEmp = h.getIdEmpleado();
          String nombreEmp = "NN";
          if (empleadosAsignados != null) {
            for (Usuario e : empleadosAsignados) {
              if (e.getId() == idEmp) {
                nombreEmp = e.getApellido() + ", " + e.getNombre();
                break;
              }
            }
          }
          if(pro.getSupervisor().getId() == idEmp){
            nombreEmp = pro.getSupervisor().getApellido() + ", " + pro.getSupervisor().getNombre();
          }
          if(administradores != null){
            for (Usuario u : administradores){
              if(u.getId() == idEmp){
                nombreEmp = u.getApellido() + ", " + u.getNombre();
                break;
              }
            }
          }
        }
      %>
      <tr class="hover:bg-gray-50">
        <td class="px-6 py-4 text-sm text-gray-900"><%= nombreEmp
%></td>

```



```

                <td class="px-6 py-4 text-sm text-gray-600"><%= h.getFecha() != null
? sdf.format(h.getFecha()) : "—" %></td>
                <td class="px-6 py-4"><%= h.getDetalle() %></td>
                <td class="px-6 py-4">
                    <span class="bg-blue-100 text-blue-800 text-sm font-semibold px-3
py-1 rounded-full">
                        <%= h.getCantidad() %> horas
                    </span>
                </td>
            </tr>
        <% } %>
        <tr class="bg-blue-50 font-semibold">
            <td colspan="3" class="px-6 py-4 text-sm text-gray-700">Total de
horas</td>
            <td class="px-6 py-4">
                <span class="bg-blue-600 text-white text-sm font-bold px-4 py-1.5
rounded-full">
                    <%= totalHoras %> horas
                </span>
            </td>
            <td></td>
        </tr>
    </tbody>
</table>
</div>
<% } %>
</div>
<!-- CONTENIDO DE COMENTARIOS -->
<div id="contentComentarios" class="p-4 sm:p-6 hidden">
    <div class="flex flex-col sm:flex-row sm:items-center sm:justify-between gap-3
mb-6">
        <h2 class="text-lg font-semibold text-gray-800">Comentarios</h2>
    </div>

    <!-- Formulario para nuevo comentario + archivos adjuntos -->
    <div class="bg-gray-50 rounded-lg p-4 mb-6">
        <form action="ComentarioServlet" method="post"
enctype="multipart/form-data">
            <input type="hidden" name="action" value="new">
            <input type="hidden" name="idTarea" value="<%= tarea != null ?
tarea.getId() : "" %>">

                <textarea name="texto" rows="3"
                    class="w-full px-3 py-2 border border-gray-300 rounded-lg
focus:ring-2 focus:ring-blue-500 focus:border-transparent resize-none"
                    placeholder="Escribe un comentario... (opcional si
adjuntas archivos)" required></textarea>

```

```

<!-- Input para adjuntar archivos (múltiples) -->
<div class="mt-3">
  <label class="block text-sm font-medium text-gray-700 mb-1">
    Adjuntar archivos (imágenes, PDFs, documentos...)
  </label>
  <input type="file"
    name="archivos"
    multiple
    accept="image/*,application/pdf,.doc,.docx,.xls,.xlsx,.txt"
    class="block w-full text-sm text-gray-500 file:mr-4 file:py-2
file:px-4 file:rounded-full file:border-0 file:text-sm file:font-semibold file:bg-blue-50
file:text-blue-700 hover:file:bg-blue-100">
  <p class="text-xs text-gray-500 mt-1">Puedes seleccionar
varios archivos (máx 10 MB total recomendado)</p>
</div>

<div class="mt-4 flex justify-end">
  <button type="submit"
    class="bg-blue-600 hover:bg-blue-700 text-white
font-medium py-2 px-6 rounded-lg transition text-sm">
    Publicar Comentario
  </button>
</div>
</form>
</div>

<!-- Lista de comentarios -->
<div class="space-y-4">
  <% if (comentarios == null || comentarios.isEmpty()) { %>
    <div class="text-center py-12 text-gray-400">
      <svg class="mx-auto h-12 w-12 mb-3" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
        <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
d="M8 12h.01M12 12h.01M16 12h.01M21 12c0 4.418-4.03 8-9 8a9.863 9.863 0
01-4.255-.949L3 20l1.395-3.72C3.512 15.042 3 13.574 3 12c0-4.418 4.03-8 9-8s9 3.582 9
8z"></path>
      </svg>
      <p class="text-sm">No hay comentarios aún</p>
    </div>
  <% } else {
    for (Comentario c : comentarios) {
      int idEmp = c.getIdEmpleado();
      String nombreEmp = "NN";
      String iniciales = "NN";
      if (empleadosAsignados != null) {
        for (Usuario e : empleadosAsignados) {
          if (e.getId() == idEmp) {
            nombreEmp = e.getApellido() + ", "
+e.getNombre();

```

```

        iniciales = e.getNombre().substring(0, 1)
+ e.getApellido().substring(0, 1);
        break;
    }
}
if(pro.getSupervisor().getId() == idEmp){
    nombreEmp = pro.getSupervisor().getApellido() + ", " +
pro.getSupervisor().getNombre();
}
if(administradores != null){
    for (Usuario u : administradores){
        if(u.getId() == idEmp){
            nombreEmp = u.getApellido() + ", " +u.getNombre();
            break;
        }
    }
}
}
}%>

```

```

<div class="bg-white border border-gray-200 rounded-lg p-4
hover:shadow-md transition">
    <div class="flex items-start gap-3">
        <div class="w-10 h-10 bg-gray-300 rounded-full flex items-center
justify-center text-white font-semibold flex-shrink-0">
            <%= iniciales %>
        </div>
        <div class="flex-1 min-w-0">
            <div class="flex flex-col sm:flex-row sm:items-center
sm:justify-between gap-1 mb-2">
                <p class="font-semibold text-gray-900 text-sm"><%= nombreEmp
%></p>
                <p class="text-xs text-gray-500"><%= c.getFecha() != null ?
sdfTime.format(c.getFecha()) : "—" %></p>
            </div>
            <p class="text-sm text-gray-700 break-words"><%= c.getTexto()
%></p>
            <!-- Archivos adjuntos -->
            <%

```

```

List<AdjuntosComentario> adjuntos =
AdjuntosComentarioDAO.obtenerPorComentario(c.getId());
if (adjuntos != null &&
!adjuntos.isEmpty()) { %>
    <div class="mt-3 flex
flex-wrap gap-2">
        <% for
(AdjuntosComentario a : adjuntos) { %>
            <!-- <a href="<%=
request.getContextPath() %><%= a.getRuta() %>" target="_blank"

```

```
class="inline-flex items-center gap-1 px-3 py-1.5 bg-gray-100 hover:bg-gray-200 rounded
text-xs font-medium text-gray-700">
```

```
class="w-4 h-4" fill="none" stroke="currentColor" viewBox="0 0 24 24">
<path
stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M15.172 7l-6.586
6.586a2 2 0 102.828 2.828l6.586-6.586"></path>
```

```
</svg>
<%=
a.getNombreOriginal().length() > 30 ? a.getNombreOriginal().substring(0,27)+"..." :
a.getNombreOriginal() %>
```

```
</a> -->
<%=
```

```
String rutaCompleta = request.getContextPath() + a.getRuta();
```

```
String tipo = a.getTipoMime() != null ? a.getTipoMime() : "";
```

```
boolean esImagen = tipo.startsWith("image/");
```

```
%>
```

```
<% if (esImagen) { %>
```

```
    <a href="<%= rutaCompleta %>" target="_blank">
```

```
        "
```

```
        class="h-32 rounded border object-cover hover:opacity-80 transition">
```

```
    </a>
```

```
<% } else { %>
```

```
    <a href="<%= rutaCompleta %>" target="_blank"
```

```
        class="inline-flex items-center gap-1 px-3 py-1.5 bg-gray-100 hover:bg-gray-200
rounded text-xs font-medium text-gray-700">
```

```
        <svg class="w-4 h-4" fill="none" stroke="currentColor" viewBox="0 0 24 24">
```

```
            <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M15.172
7l-6.586 6.586a2 2 0 102.828 2.828l6.586-6.586"></path>
```

```
        </svg>
```

```
        <%= a.getNombreOriginal().length() > 30 ? a.getNombreOriginal().substring(0,27)+"..." :
a.getNombreOriginal() %>
```

```
    </a>
```

```
<% } %>
```

```
<% } %>
```

```
</div>
```

```
<% } %>
```

```
<% if (usuarioActual != null && usuarioActual.getId() ==
c.getIdEmpleado()) { %>
```

```
    <button onclick="eliminarComentario(<%= c.getId() %>)"
```

```
        class="text-xs text-red-600 hover:text-red-800 mt-2">
```

```
        Eliminar
```

```
    </button>
```

```
<% } %>
```

```
</div>
```

```
</div>
```

```
</div>
```

```

        <% }
    } %>
</div>
</div>
</div>
<% } else { %>
<!-- Mensaje para quien no tiene permiso -->
<div class="bg-white rounded-lg shadow p-8 text-center">
    <svg class="mx-auto h-16 w-16 text-gray-400 mb-4" fill="none" stroke="currentColor"
viewBox="0 0 24 24">
        <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M12
15v2m-6 4h12a2 2 0 02-2v-6a2 2 0 00-2-2H6a2 2 0 00-2 2v6a2 2 0 002 2zm10-10V7a4 4 0
00-8 0v4h8z"></path>
    </svg>
    <h3 class="text-lg font-medium text-gray-900 mb-2">Acceso restringido</h3>
    <p class="text-gray-600">Solo los administradores y las personas asignadas a esta
tarea pueden ver esta información.</p>
</div>
<% } %>

</div>
<!-- MODAL ASIGNAR EMPLEADO -->
<div id="modalAsignarEmpleado" class="hidden fixed inset-0 bg-black bg-opacity-50 flex
items-center justify-center z-50 p-4">
    <div class="bg-white rounded-lg shadow-xl max-w-md w-full max-h-[90vh]
overflow-y-auto">
        <div class="p-4 sm:p-6">
            <div class="flex items-center justify-between mb-4">
                <h3 class="text-lg font-semibold text-gray-900">Asignar Persona</h3>
                <button onclick="toggleModal('modalAsignarEmpleado')" class="text-gray-400
hover:text-gray-600">
                    <svg class="w-6 h-6" fill="none" stroke="currentColor" viewBox="0 0 24 24">
                        <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M6
18L18 6M6 6l12 12"></path>
                    </svg>
                </button>
            </div>
            <form action="TareaServlet?action=asignar&idTarea=<%= tarea.getId() %>"
method="post">
                <input type="hidden" name="action" value="asignar">
                <input type="hidden" name="idTarea" value="<%= tarea != null ? tarea.getId() : ""
%>">
                <!-- Barra de búsqueda -->
                <input
                    type="text"
                    id="buscadorEmpleado"
                    placeholder="Buscar empleado..."
                    class="w-full mb-3 px-3 py-2 border rounded-lg focus:ring focus:ring-blue-200"

```

```

        onkeyup="filtrarEmpleados()"
    />
    <!-- Lista de empleados -->
    <div id="lista-empleados" class="border rounded-lg px-4 py-3 max-h-40
overflow-y-auto">
        <% if (empleadosDisponibles != null) {
            for (Usuario u : empleadosDisponibles) {
                boolean seleccionado = empleadosAsignados != null &&
                    empleadosAsignados.stream().anyMatch(us -> us.getId() == u.getId());
        %>
            <label class="flex items-center space-x-2 mb-2 cursor-pointer empleado-item">
                <input type="checkbox"
                    name="usuarios"
                    value="<%= u.getId() %>"
                    <%= seleccionado ? "checked" : "" %>
                    class="h-4 w-4 text-blue-600 focus:ring-blue-500 border-gray-300
rounded">
                    <span class="empleado-nombre">
                        <%= u.getNombre() %> <%= u.getApellido() %>
                    </span>
                </label>
            <% } } %>
        </div>
        <div class="flex flex-col sm:flex-row gap-2 justify-end mt-4">
            <button type="button" onclick="toggleModal('modalAsignarEmpleado')"
                class="px-4 py-2 text-gray-700 bg-gray-200 hover:bg-gray-300 rounded-lg
transition">
                Cancelar
            </button>
            <button type="submit"
                class="px-4 py-2 bg-blue-600 hover:bg-blue-700 text-white rounded-lg
transition">
                Asignar
            </button>
        </div>
    </form>
</div>
</div>
</div>
<jsp:include page="../../../horasTrabajadas/formulario.jsp" />
<jsp:include page="../../../tareas/formulario.jsp" />

```